📋 SYSTEM DOCUMENTATION

Missing Child Identification System — Overall Workflow

An AI-powered facial recognition system for identifying and locating missing children using InsightFace ArcFace embeddings and PostgreSQL pgvector similarity search.

Generated: March 26, 2026 | Version: Production (InsightFace/ONNX)

System Architecture

Frontend
React + Vite + TailwindCSS
Served via Nginx
`:80`

Backend API
FastAPI + InsightFace
ONNX Runtime (CPU)
`:8000`

Database
PostgreSQL 16
+ pgvector extension
`:5432`

All three services are orchestrated via `docker-compose.yml`. The frontend proxies all `/api/*` requests to the backend container.

End-to-End Workflow

1

🚀 System Startup

PostgreSQL starts and becomes healthy. The FastAPI backend then initializes: creates **children** and **search_logs** DB tables, loads the **InsightFace buffalo_l** model (SCRFD + ArcFace) into memory, and mounts the uploads directory.

```
docker-compose up --build → db (postgres + pgvector) starts → healthcheck passes → backend starts → init_db() creates tables → InsightFace buffalo_l model loaded into RAM → System ready at http://localhost:8000
```

2

🔐 Authentication (Admin Login)

Admin users log in at `/login`. The frontend sends credentials to `POST /api/token` and receives a JWT. The token is stored in `localStorage` via `AuthContext`. **ProtectedRoute** components

guard the Dashboard, Register, Search, and Reports pages — redirecting unauthenticated users to `/login`.

3

📝 Register a Missing Child

An admin fills in the child's details and uploads a face photo on the **Register** page. The form is submitted as `multipart/form-data` to the backend, which runs the full AI pipeline:

```
// POST /api/register/ (fields: name, age, gender, location, phone, email, file) 1. Validate gender & image format 2. Decode image → OpenCV BGR array 3. InsightFace.get(image) → detect faces 4. Reject if 0 faces or >1 face 5. Extract 512-dim ArcFace embedding → L2 normalize 6. Save image to /uploads/<uuid>.jpg 7. INSERT child + embedding (vector(512)) → PostgreSQL → Returns: ChildOut JSON
```

4

🔍 Search for a Found Child

Anyone uploads a photo of a found child on the **Search** page. The backend generates an embedding and performs a vector similarity search against all registered missing children:

```
// POST /api/search/ (field: image) 1. Validate & decode image 2. extract_embeddings() → 512-dim ArcFace vector 3. Save query image to /uploads/queries/ -- pgvector SQL -- SELECT id, name, ... FROM children WHERE status='missing' ORDER BY arcface_embedding <=> :query_vec ASC LIMIT top_k -- distance < threshold 4. Convert cosine distance → confidence% (sigmoid map) 5. Log search entry → search_logs table → Returns: SearchResult {match_found, matches[], message}
```

5

🚨 Automated Alert System

If the best match confidence reaches **≥ 85%**, the system sends real-time alerts to the registered family contact — preventing duplicate alerts via the `alert_sent` flag on the child's record.

Email Alert
Sent to `contact_email` via SMTP if configured.

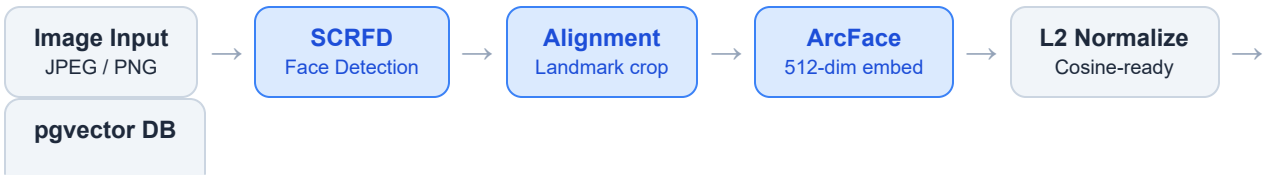
SMS Alert
Sent to `contact_number` via Twilio if configured.

6

📊 Dashboard & Reports

Dashboard displays system statistics (total missing children, total searches, matches found). **Reports** lists all search log entries with match details, confidence scores, timestamps, and IP addresses via `GET /api/reports/`.

🧠 AI Face Recognition Pipeline



Store / Search

Model: **InsightFace buffalo_l** running on ONNX Runtime (CPU). Initialized once at startup. Both Registration and Search run through the same pipeline — ensuring embedding consistency.

Key File Map

FILE	LAYER	PURPOSE
`docker-compose.yml`	Infrastructure	Orchestrates DB, Backend, Frontend containers
`backend/main.py`	Backend Entry	FastAPI app setup, CORS, routers, SPA fallback
`backend/config.py`	Config	Env settings: DB URL, thresholds, upload directory
`backend/models.py`	ORM	`Child` (with 512-d vector), `SearchLog` tables
`backend/schemas.py`	Validation	Pydantic I/O schemas for API request/response
`backend/auth.py`	Auth	JWT token generation & verification
`routers/register.py`	API	Child registration endpoint — full pipeline
`routers/search.py`	API	Face search + alert trigger logic
`services/face_recognition.py`	AI	InsightFace init + `extract_embeddings()`
`services/similarity.py`	AI	pgvector cosine search + confidence scoring
`frontend/src/App.jsx`	Frontend	React Router: Login, Dashboard, Register, Search, Reports
`frontend/src/context/AuthContext.jsx`	Frontend	JWT state management across the app

Key Design Decisions

AI / Model

InsightFace over DeepFace / TensorFlow

Eliminated download failures; lightweight ONNX runtime with no GPU needed. Consistent performance across environments.

Database

pgvector for similarity search

Native Postgres vector search — no separate vector DB (e.g., Pinecone/Weaviate) required. Simplifies deployment.

AI / Accuracy

512-dim ArcFace embeddings

Industry-standard embeddings with large-scale face recognition benchmarks. High discriminability reduces false positives.

Access Control

Public search, protected admin pages

Anyone can report or search for a missing child without login. Dashboard and Reports require admin JWT authentication.

Alerts

Alert threshold at 85% confidence

Sigmoid-mapped confidence scoring avoids false positive email/SMS alerts to family contacts. One-time alert enforced via `alert_sent` flag.

Security

Rate limiting on all API endpoints

SlowAPI + IP-based rate limiting prevents abuse of the open search endpoint and brute-force attacks on authentication.